

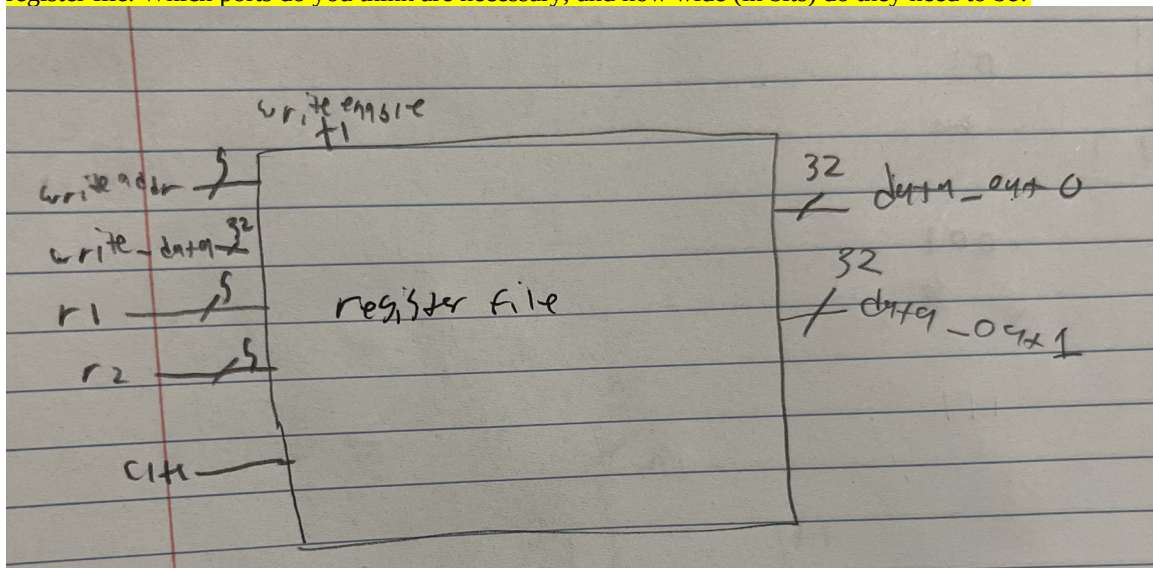
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

Student Name Ryan Hoefer

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

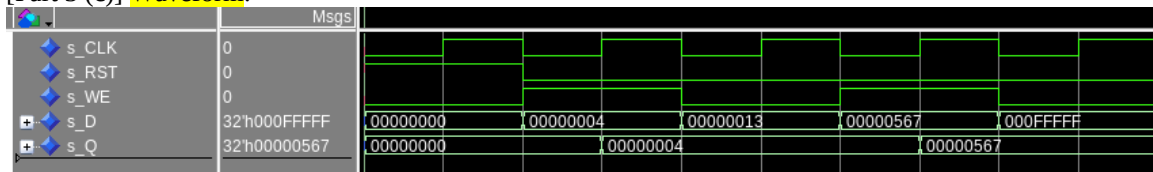
[Part 3 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?



[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.

ok

[Part 3 (c)] Waveform.

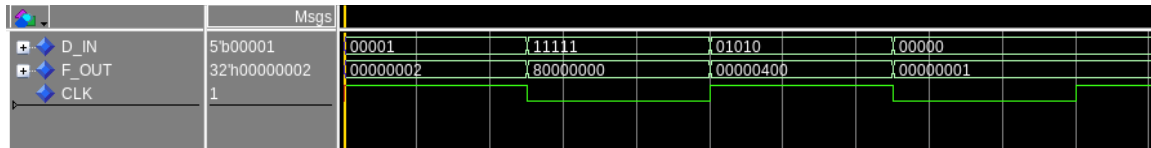


This correctly stores the values 4 and 567 when the we is enabled and doesn't store 13 and FFFFF when it isn't

[Part 3 (d)] What type of decoder would be required by the RISC-V register file and why?

A 5 to 32 decode because there are 32 registers

[Part 3 (e)] Waveform.

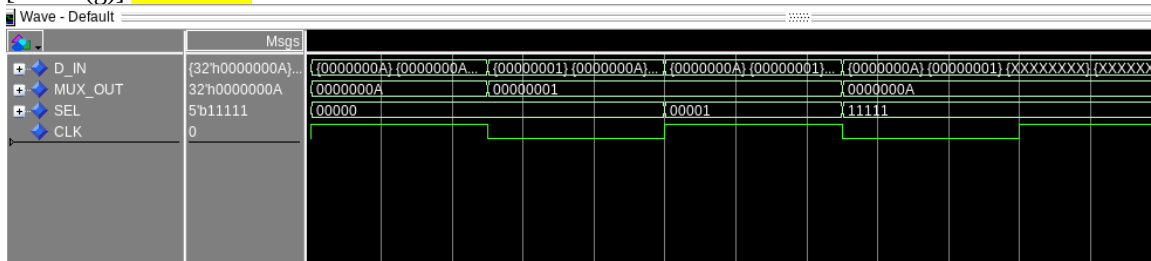


This correctly outputs to the line it should based on what the in is

[Part 3 (f)] In your write-up, describe and defend the design you intend on implementing for the next part.

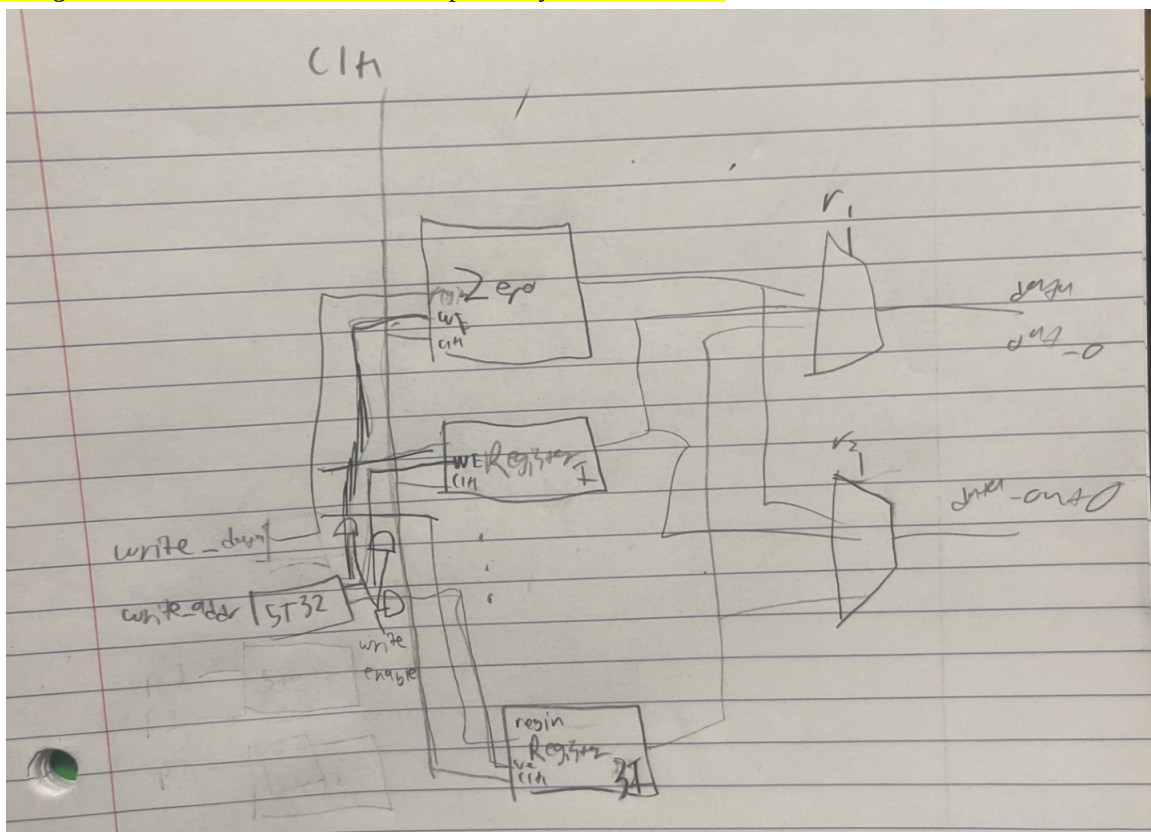
I used a custom type to use a 32 wide array of 32 bit signals, this would reduce the boilerplate needed and make it more readable. Also this allows me to re use this component later on if I need to as well. Dataflow would be 32 lines long and be a lot of code to type.

[Part 3 (g)] Waveform.

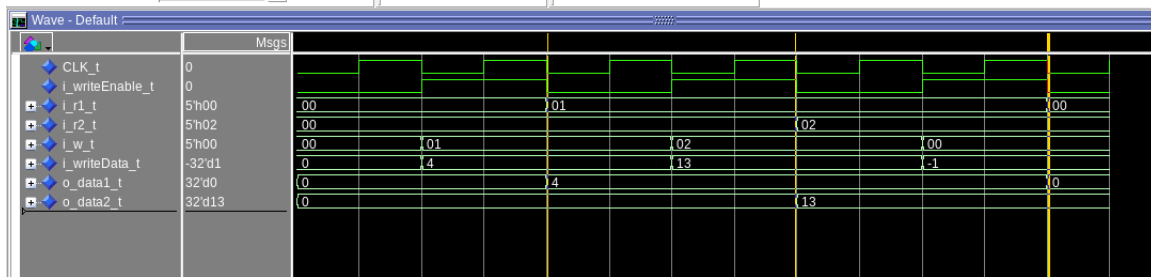


This one correctly passed what ever mux value was selected based on the select input

[Part 3 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.

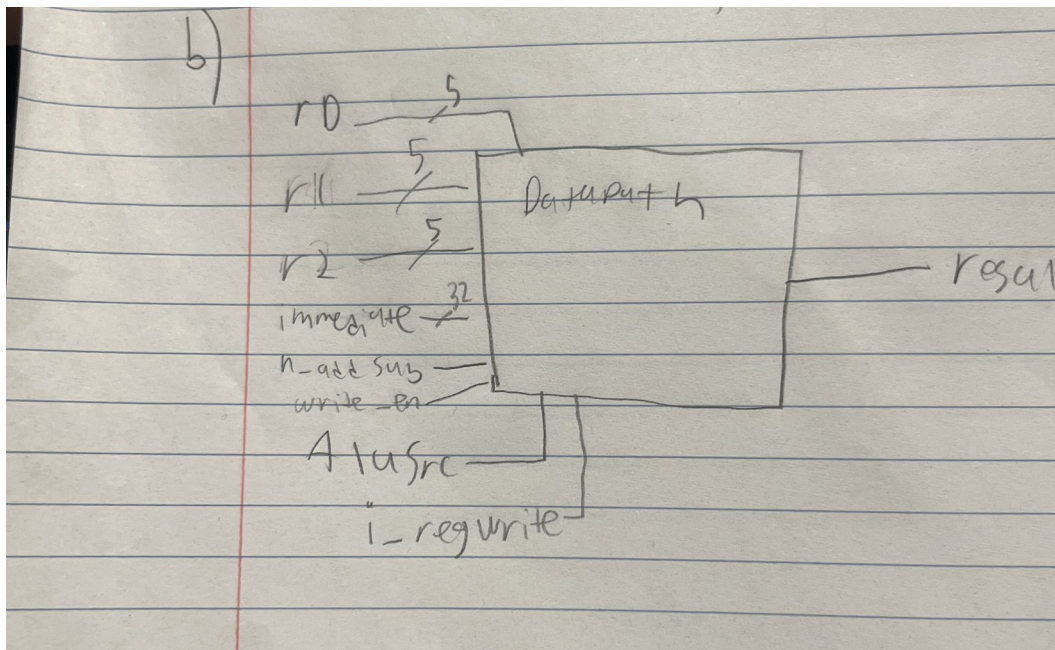


[Part 3 (i)] Waveform.

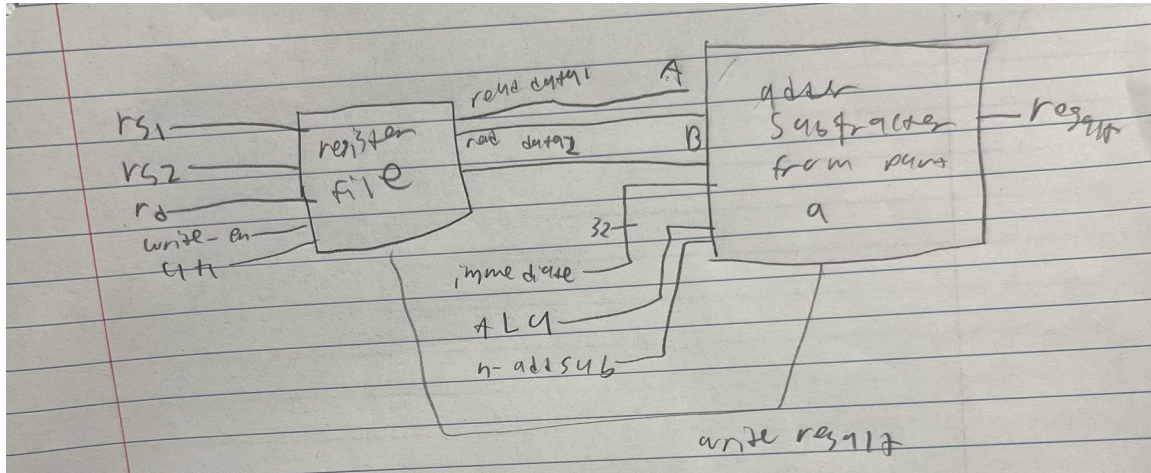


- Test 1
 - read 0 from x0 on both ports
- Test 2
 - Store 4 in x1 and read
- Test 3
 - write 13 to x2 then read x1 and x2
- Test 4
 - write to x0 and read making sure it is still 0

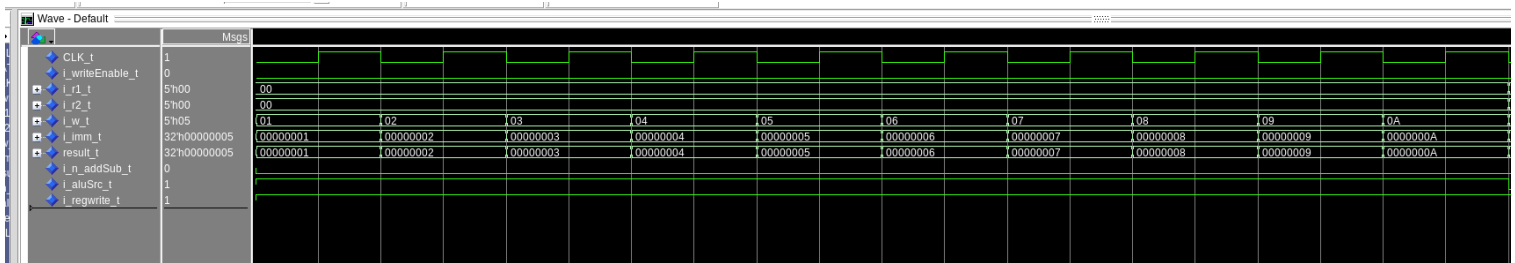
[Part 4 (b)] Draw a symbol for this RISC-V-like datapath.



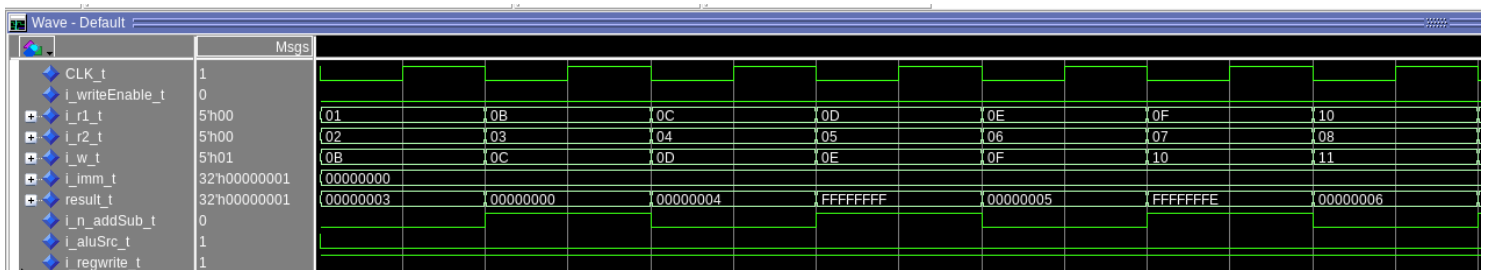
[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final state of the register file should be and provide a mapping of the registers used in your drawing to the register names.



This is the tests loading data into registers

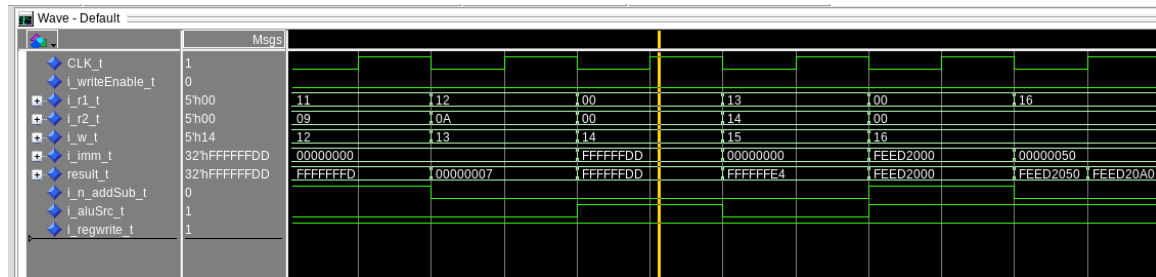


This is the assembly code here

```

add x11, x1, x2 # x11 = x1 + x2
sub x12, x11, x3 # x12 = x11 - x3
add x13, x12, x4 # x13 = x12 + x4
sub x14, x13, x5 # x14 = x13 - x5
add x15, x14, x6 # x15 = x14 + x6
sub x16, x15, x7 # x16 = x15 - x7
add x17, x16, x8 # x17 = x16 + x8

```



this is this code

```
sub x18 , x17 , x9 # x18 = x17 - x9
add x19 , x18 , x10 # x19 = x18 + x10
addi x20 , zero , -35 # Place "-35" in x20
add x21 , x19 , x20 # x21 = x19 + x20
lui x22 , 0xFEED2 # Place "0xFEED2000" in x22
addi x22 , x22 , 0x050 # Complete loading x22 with a large immediate
```

The final state of the register file should be the foollowing with mapping being the associated binary value to be x(value)

```
x0-0
x1-1
x2-2
x3-3
x4-4
x5-5
x6-6
x7-7
x8-8
x9-9
x10-10
x11-3
x12-0
x13-4
x14-(-1)
x15-5
x16-(-2)
x17-6
x18-(-3)
x19-7
x20-(-35)
x21-(-28)
x22-0xFEED2050
x23 through x31 is undefined
```

[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

Clk: you need a clock because the data is stored synchronously so it needs a clock edge to change.
Addr: this is the memory location for both read and write. This selects each word not each individual byte

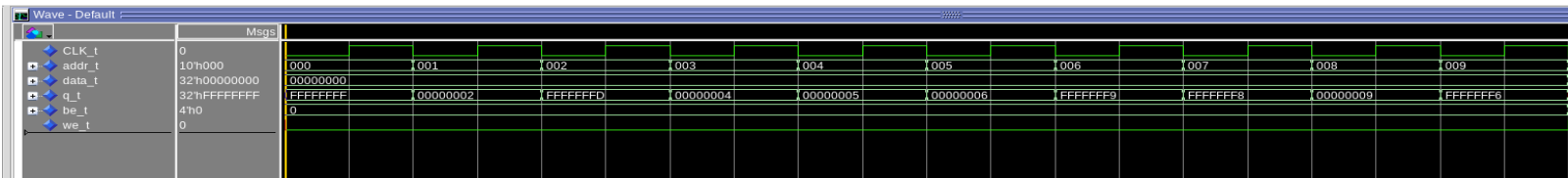
data: this is what will be written to memory. This happens on a rising edge.

be: this is the byte enable, which determines which of the four bytes is written. Examples are 1111 is all bytes are written but 1100 means only bytes 3 and 2 are written.

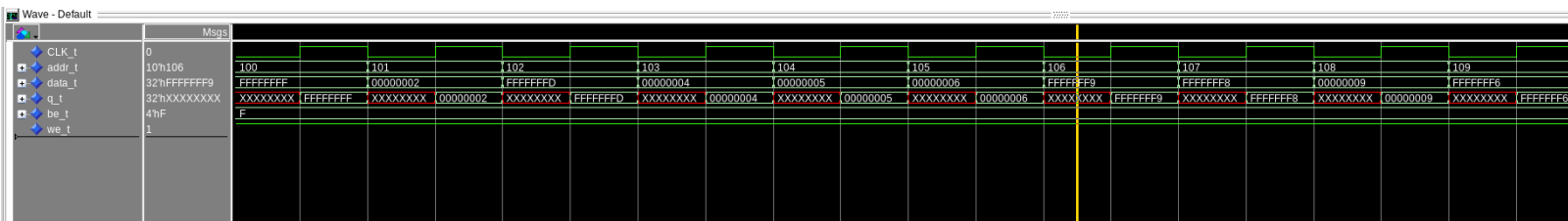
We: this is a global write enable so it will control if all the bytes are allowed to be written. This will be checked on the rising clock edge.

q: the read output of the memory. This is done asynchronously so you don't need the clock edge to access this.

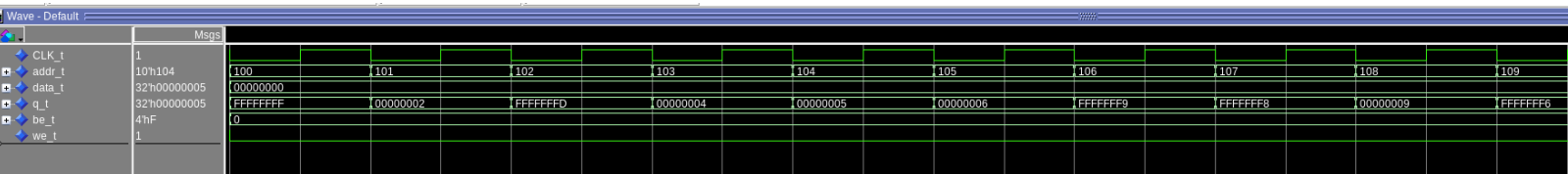
[Part 5 (c)] Waveforms and hex file.



This reads the initial 10 values(part b)



this writes those values back into memory(part c)



this reads it back(part d)

```
tb_dmem.vhd  tb_datapath.vhd  datapath.vhd  register_N.vhd  mem.vhd  dmem.hex  src > dmem.hex
1 // File Format:
2 //
3 // The text file can contain whitespace characters, comments (C-style),
4 // and binary ($readmemb) or hexadecimal ($readmemh) data values. Both
5 // types of data values can contain x or X, z or Z, and the underscore
6 // character.
7 //
8 // The data values are assigned to memory words from left to right,
9 // beginning at start_addr or the left array bound (the default). The
10 // next address to load may be specified in the file itself using @hhhhh,
11 // where h is a hexadecimal character. Spaces between the @ and the address
12 // character are not allowed. It shall be an error if there are too many
13 // words in the file or if an address is outside the bounds of the array.
14 //
15 @000
16 FFFFFFFF
17 00000002
18 FFFFFFFD
19 00000004
20 00000005
21 00000006
22 FFFFFFF9
23 FFFFFFF8
24 00000009
25 FFFFFFF6
26
```

[Part 6 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended? What are the RISC-V instructions that require some value to be appended with zero?

Sign extended: lbu, ori, lb, lw, sw

zero extended: lui, lhu

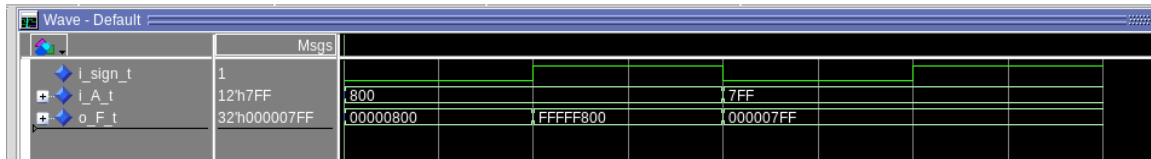
appended with zero: blt, bltu, bne, jal

[Part 6 (b)] what are the different “extender” components that would be required by a RISC-V processor implementation (i.e., number of bits of input and sign vs zero extension vs zero appending)?

12 to 32 sign extender, 8 to 32 sign extender

12 to 32 zero extender, 16 to 32 zero extender

[Part 6 (d)] Waveform.



test 1: zero extends the 12 bit number where 1 is sign

test 2: sign extends the 12 bit number where 1 is sign

test 3: zero extends the 12 bit number where 0 is sign

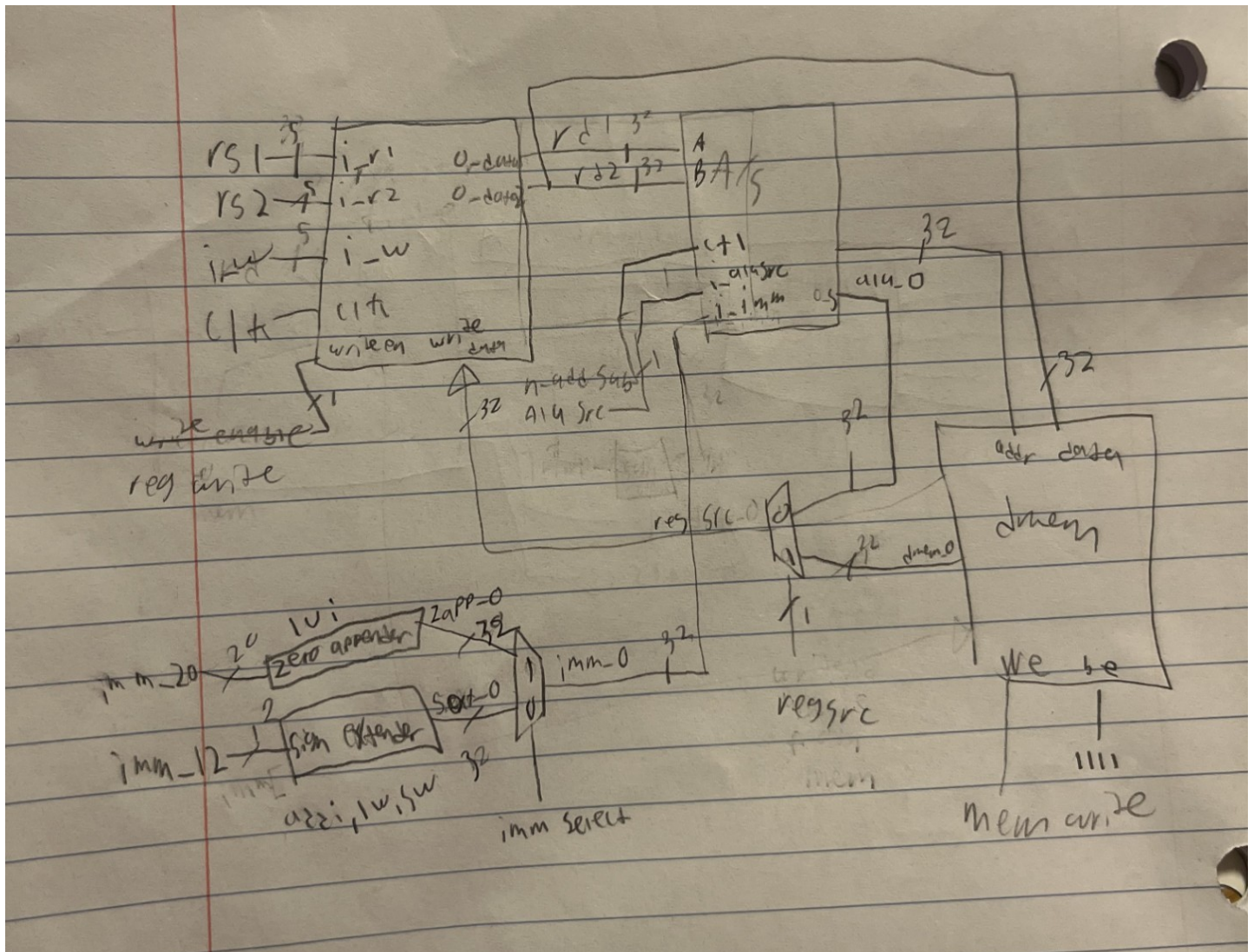
test 4: sign extends the 12 bit number where 0 is sign

[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

It would need to have a mem write enable that would connect to the we control bit in the mem file. It would also need to have the be field set to 1111 to enable all 4 bytes because risc-v doesn't support enabling or disabling individual bytes

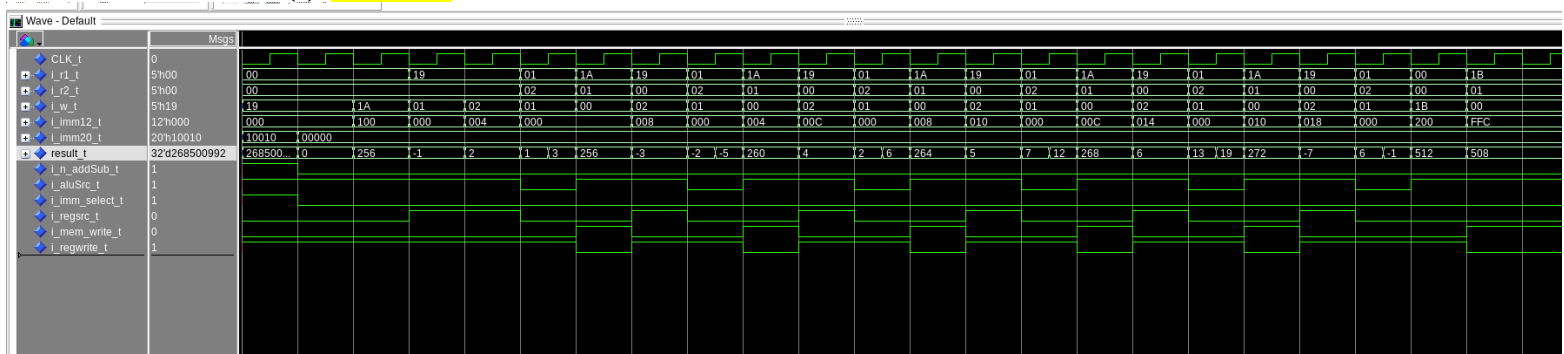
we would also need another control bit to select what the register file would receive between the memory and the alu.

[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



This is the top level schematic of the datapath

[Part 7 (c)] **Waveform.**



This implements the assembly code given, the highlighted result_t signal is the output from the mux to the register file that has control regsrc.

[Part 7 (d)] **Compare the Questasim waveform/memory list against the RARs output/memory list.**

Text Segment		Address	Code	Basic	Source
	Bkpt				
		0x00003000	0x10010eb7	lui x25,0x00010010	8: lui x25 , 0x10010 # Initialize x25 to 0 x1001...
		0x00003004	0x00000c93	addi x25,x0,0	9: addi x25 ,zero ,0 # Load &A into x25 , assumi...
		0x00003008	0x10000d13	addi x26,x0,0x00000100	10: addi x26 ,zero ,256 # Load &B into x26 , assu...
		0x0000300c	0x000ca083	lw x1,0(x25)	11: lw x1 , 0(x25) # Load A[0] into x1
		0x00003010	0x004ca103	lw x2,4(x25)	12: lw x2 , 4(x25) # Load A[1] into x2 ,
		0x00003014	0x002080b3	add x1,x1,x2	13: add x1 , x1 , x2 # x1 = x1 + x2
		0x00003018	0x001d2023	sw x1,0(x26)	14: sw x1 , 0(x26) # Store x1 value into B[0]
		0x0000301c	0x008ca103	lw x2,8(x25)	15: lw x2 , 8(x25) # Load A[2] into x2
		0x00003020	0x002080b3	add x1,x1,x2	16: add x1 , x1 , x2 # x1 = x1 + x2
		0x00003024	0x001d2223	sw x1,4(x26)	17: sw x1 , 4(x26) # Store x1 value into B[1]
		0x00003028	0x00cca103	lw x2,12(x25)	18: lw x2 , 12(x25) # Load A[3] into x2
		0x0000302c	0x002080b3	add x1,x1,x2	19: add x1 , x1 , x2 # x1 = x1 + x2

Data Segment		Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
		0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x0000000e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x00000010	0x00000001	0xffffffff	0x00000002	0x00000007	0x0000000d	0x00000000	0x00000000	0x00000000
		0x00000012	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x00000014	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x00000016	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x00000018	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x0000001a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x0000001c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
		0x0000001e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000087	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000081	XXXXXXXX	XXXXXXXX	00000006	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000007b	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000075	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000006f	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000069	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000063	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000005d	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000057	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000051	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000004b	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000045	XXXXXXXX	0000000d	00000007	00000002	FFFFFFFE	00000001	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000003f	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000039	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000033	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000002d	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000027	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000021	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000001b	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000015	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000000f	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000009	FFFFFFF6	00000009	FFFFFFF8	FFFFFFF9	00000006	00000005	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000003	00000004	FFFFFFFD	00000002	FFFFFFF7	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX

This looks to be all good and matching with the questasim starting at 40 and rars atarting at 100. This is because my implementation uses word addressing and RARS uses byte addressing. The word array sequentially follow and we have the stray 6 a little ways away.